COMINDIR03: Relógios Lógicos e Vetoriais sobre Partições

Documento de Apoio e Análise de Código

Implementação e Engenharia de Sistemas Distribuídos

26 de outubro de 2026

Conteúdo

1 Resumo da Mecânica dos Atores **2**

2 Enquadramento Teórico **2**

3 O Motor Matemático: `VectorClock` **2**
3.1 Operações Base 3
3.2 Comparação Causal 3
3.3 Serialização e Desserialização 3

4 O Produtor (`Producer`) **4**

5 O Líder de Partição (`PartitionLeader`) **4**
5.1 A Secção Crítica 4
5.2 Recuperação de Estado 5

6 O Processador de Alarmes (`AlarmProcessor`) **5**
6.1 Deduplicação 5

7 A Réplica (`PartitionReplica`) **6**
7.1 Retenção de Mensagens 6
7.2 Recuperação TCP 6

8 Ferramentas de Análise **7**
8.1 PartitionCheck 7
8.2 CausalCheck 7

9 As Falhas Deliberadas (Ponto 8) **7**

10 Reflexão Crítica (Ponto 10) **8**

1 Resumo da Mecânica dos Atores

Antes de analisar o código, é crucial perceber o papel de cada um dos cinco atores no ecossistema e como o conhecimento causal flui entre eles:

Produtor É a origem externa de dados. Não tem memória do sistema, logo não observa eventos de outras partições. Por isso, envia sempre os relógios a zero ($L = 0$ e $V = (0, \ldots, 0)$) e usa o `Partitioner` para encaminhar a leitura para o líder correto.

Líder de Partição É o único criador de eventos locais. Recebe a leitura, absorve o conhecimento das mensagens (via `merge` e `max`), incrementa a sua própria componente (via `tick`) e atribui o número de sequência (`seq`). Grava o estado no ficheiro para sobreviver a reinícios.

Processador de Alarmes É a ponte de causalidade. Subscreve todos os tópicos e absorve o conhecimento global (faz `merge` e `max`, mas **nunca** faz `tick`). Se a temperatura for superior a 28.0°C, cria um novo sensor (`ALARM-...`) e envia-o para outra partição, transportando o vetor que absorveu e criando assim uma relação $a \to b$.

Réplicas São os arquivos passivos. Aplicam os registos estritamente pela ordem do `seq` de cada partição. Apesar de guardarem o L e o V no disco, **ignoram-nos para tomar decisões**. Não têm entrega causal: se receberem um alarme antes da leitura que o originou, aplicam o alarme.

CausalCheck É o auditor offline. Não corre em tempo real. Lê os ficheiros gerados e usa a matemática dos vetores para provar a relação de causalidade entre partições. É ele que demonstra que o relógio de Lamport falha ao tentar ordenar eventos que são, na realidade, concorrentes.

2 Enquadramento Teórico

O objetivo central desta ficha é determinar relações de causalidade entre eventos que ocorrem em partições diferentes. Como os líderes não comunicam entre si diretamente, o sistema introduz um *processador* que observa uma partição e escreve noutra, criando dependências. Para as representar, usamos dois mecanismos:

- **Relógio de Lamport (L)**: Um contador inteiro que impõe uma ordem total arbitrária aos eventos. Garante que se $a \to b$, então $L(a) < L(b)$, mas não o contrário.
- **Relógio Vetorial (V)**: Um array de tamanho N (número de partições) que caracteriza a causalidade. Se $V(a) < V(b)$, então $a \to b$. Se forem incomparáveis, os eventos são concorrentes ($a \parallel b$).

3 O Motor Matemático: `VectorClock`

A classe `VectorClock` é o cérebro matemático partilhado por todos os projetos.

3.1 Operações Base

```
    public void merge(VectorClock other) {
      for (int i = 0; i < v.length; i++) {
        v[i] = Math.max(v[i], other.v[i]);
      }
    }

    public void tick(int k) {
      v[k]++;
    }

```

Explicação: O `merge` absorve o conhecimento dos outros líderes (ficando sempre com o valor máximo de cada posição). O `tick(k)` incrementa a posição do próprio líder, simbolizando um novo evento local.

3.2 Comparação Causal

```
    public Order compare(VectorClock other) {
      boolean menor = false, maior = false;
      for (int i = 0; i < v.length; i++) {
        if (v[i] < other.v[i]) menor = true;
        if (v[i] > other.v[i]) maior = true;
      }
      if (menor && maior) return Order.CONCURRENT;
      if (menor) return Order.BEFORE;
      if (maior) return Order.AFTER;
      return Order.EQUAL;
    }

```

Explicação: Se houver uma componente maior e outra menor, os vetores são incomparáveis (`CONCURRENT`). Se só houver componentes menores, o evento atual aconteceu antes (`BEFORE`).

3.3 Serialização e Desserialização

```
    public static VectorClock parse(String s, int n) {
      String[] f = s.trim().split(",");
      if (f.length != n) {
        throw new IllegalArgumentException("Vetor com dimensao " + f.
   length + ", esperado " + n);
      }
      long[] v = new long[n];
      for (int i = 0; i < n; i++) {
        v[i] = Long.parseLong(f[i]);
      }
      return new VectorClock(v);
    }

```

Explicação: O `parse` é vital para ler a string que vem do IPFS. Se o produtor enviar $N = 4$ mas o líder esperar $N = 3$, este método lança uma exceção e o líder rejeita a mensagem.

Pergunta do Agente: O que é o `copy()` e porquê usá-lo?

A classe `VectorClock` é mutável. Se passarmos a referência original para um ficheiro ou para o `publish`, as chamadas futuras a `merge` ou `tick` alterariam o array partilhado, corrompendo o registo histórico. O `copy()` cria um *snapshot* (clone) garantindo que a mensagem transporta o estado exato daquele milissegundo.

4 O Produtor (`Producer`)

O produtor é o "zero absoluto". Ele não tem memória do sistema, não observa eventos e não incrementa relógios.

```
    private static int publicar(IpfsPubSub ps, Partitioner part, String sensor, double temp) throws Exception {
        int k = part.of(sensor); // Descobre a particao de destino
        VectorClock vetorZero = new VectorClock(part.count());
        ps.publish(Wire.topicIn(k), Wire.in(sensor, temp, 0L, vetorZero));
        return k;
    }

```

Explicação: Ele envia sempre $L = 0$ e $V = (0, 0, 0)$. O produtor usa o `Partitioner` para calcular o tópico de destino (ex: p1). Se a rede do IPFS bloquear, adiciona-se um `Thread.sleep(5)` no ciclo da rajada para evitar *rate limiting*.

5 O Líder de Partição (`PartitionLeader`)

É o criador de eventos. Recebe um `IN`, atribui um `seq` e gera o seu próprio tempo lógico.

5.1 A Secção Crítica

```
    synchronized (LOCK) {
        // 1. Calcular estado temporario
        long novoL = Math.max(lamport, L_msg) + 1;
        VectorClock novoV = vector.copy();
        novoV.merge(V_msg);
        novoV.tick(k);
        long novoSeq = ultimoSeq + 1;

        // 2. Verificar invariante
        if (novoV.get(k) != novoSeq) { throw new IllegalStateException();
    }

        // 3. Persistir e Confirmar
        r = SensorRecord.now(novoSeq, sensor, temp, novoL, novoV);
        file.append(r);

        lamport = novoL;
        vector = novoV;
        ultimoSeq = novoSeq;
    }

```

Explicação: O líder usa o `max` para o Lamport e o `merge` + `tick` para o vetor. Tudo acontece atomicamente dentro do `synchronized` para evitar que duas mensagens simultâneas leiam o mesmo `ultimoSeq` e gerem o mesmo número de sequência.

5.2 Recuperação de Estado

```
    SensorRecord ultimoRegisto = file.last();
    if (ultimoRegisto != null) {
      ultimoSeq = ultimoRegisto.seq;
      lamport = ultimoRegisto.lamport;
      vector = ultimoRegisto.vector.copy();
    } else {
      ultimoSeq = 0; lamport = 0; vector = new VectorClock(n);
    }

```

Explicação: Se o líder crashar e reiniciar, ele lê a última linha do ficheiro e recupera não só o `seq`, mas também os relógios L e V. Se partisse de zero, geraria eventos no "passado"e quebraria a causalidade.

Pergunta do Agente: Se a escrita falhar, o que acontece?

O código calcula os novos relógios em variáveis temporárias (`novoL`, `novoV`, `novoSeq`). Se o `file.append(r)` lançar uma exceção (falha de I/O), o estado real do líder (`lamport`, `vector`, `ultimoSeq`) não é alterado. Apenas após o sucesso da escrita é que os valores temporários são comitados para o estado do líder.

6 O Processador de Alarmes (`AlarmProcessor`)

É a ponte que liga as partições e cria dependências causais.

```
    synchronized (lock) {
      lamport = Math.max(lamport, r.lamport);
      vector.merge(r.vector); // Absorve, sem tick!

      if (r.getTemperature() >= Wire.LIMIAR_ALARME) {
        String alarmSensor = "ALARM-" + r.getSensorId();
        int targetPartition = part.of(alarmSensor);
        alarmMsg = Wire.in(alarmSensor, r.getTemperature(), lamport,
  vector.copy());
      }
    }
    if (alarmMsg != null) pubsub.publish(Wire.topicIn(targetPartition),
  alarmMsg);

```

Explicação: O processador não tem `+1` no Lamport nem `tick` no vetor, porque não cria eventos próprios, apenas transporta conhecimento. Ele ignora sensores que começam por `ALARM-` para evitar ciclos infinitos.

6.1 Deduplicação

```
    private final Set<Long>[] seenSeqs = new Set[n];
    // No construtor:
    this.seenSeqs[i] = ConcurrentHashMap.newKeySet();

    // No receber:
    if (!seenSeqs[k].add(r.getSeq())) return;

```

Explicação: Como as threads do IPFS correm em paralelo, usa-se um `ConcurrentHashMap.newKeySet(` (um conjunto thread-safe). Se a mensagem chegar duplicada, o `.add()` devolve `false` e o processador sai sem gerar dois alarmes.

7 A Réplica (`PartitionReplica`)

A réplica é passiva. Guarda os dados, garante a ordem local por `seq`, mas **ignora os relógios L e V** para tomar decisões (ausência de entrega causal).

7.1 Retenção de Mensagens

```
    void aplicar(SensorRecord r) {
      long seq = r.getSeq();
      if (seq <= ultimo || retidos.containsKey(seq)) {
        // Duplicado: ignorar
      } else if (seq == ultimo + 1) {
        escrever(r, false);
        escreverRetidosEmSequencia();
      } else {
        retidos.put(seq, r); // Fora de ordem: retem
      }
    }

```

Explicação: Se a réplica receber o `seq=7` mas faltar o `6`, guarda o 7 num `TreeMap` (que mantém a ordem). Quando o 6 chegar, aplica-o e depois aplica o 7.

7.2 Recuperação TCP

Se a memória dos retidos encher (acima de `LIMITE_RETIDOS`), a réplica liga-se ao líder por TCP e pede os registos em falta.

```
    out.println("PEDIDO " + de + " " + ate);
    // ... le os registos ...
    SensorRecord r = SensorRecord.fromLine(linha, n);
    aplicar(r);

```

Pergunta do Agente: A réplica faz entrega causal?

Não. A secção 2.1 da ficha proíbe a entrega causal. Se receber um alarme de p1 antes da leitura de p0 que o originou, aplica o alarme (porque o seq de p1 está correto), ignorando o vetor. O V só é guardado no ficheiro para auditoria posterior.

8 Ferramentas de Análise

8.1 PartitionCheck

Lê os ficheiros offline e valida:

- Se os sensores estão na partição correta.
- Se o `seq` é contínuo (1, 2, 3...) sem saltos.
- Se existem $R = 2$ réplicas por partição.
- Se a réplica é igual ao líder, um "prefixo"(atrasada) ou divergente.

8.2 CausalCheck

É o auditor matemático. Compara todos os pares de partições diferentes.

```
    if (ordem == VectorClock.Order.BEFORE) {
      aAntesB++;
      if (a.r.lamport >= b.r.lamport) violacoesLamport++;
    } else if (ordem == VectorClock.Order.CONCURRENT) {
      concorrentes++;
      if (a.r.lamport != b.r.lamport) concorrentesLDistinto++;
    }

```

Pergunta do Agente: Porque cerca de 95% dos pares concorrentes têm L distinto?

Como o relógio de Lamport impõe uma ordem total arbitrária, ele atribui valores diferentes a eventos que na realidade aconteceram em paralelo. A alta percentagem medida no CausalCheck prova empiricamente que a `vista-lamport.txt` não serve para inferir causalidade.

9 As Falhas Deliberadas (Ponto 8)

1. **Líder sem `merge`:** A componente perde a dependência. O `CausalCheck` classifica os alarmes como *concorrentes* em vez de causais.
2. **Líder sem `L_msg`:** O Lamport local é inferior ao evento original. O `CausalCheck` deteta uma *violação explícita da condição de Lamport.*
3. **Reinício a zero:** O invariante $V[k] = seq$ quebra (ex: $V[k] = 1$ e $seq = 101$). O `CausalCheck` e o `PartitionCheck` detetam inversão temporal.
4. **Processador sem `copy()`:** Ocorre uma condição de corrida, enviando um vetor "futuro"que já foi alterado por outra thread.
5. **Produtor com $N = 4$:** O líder espera $N = 3$. O `VectorClock.parse()` lança exceção e a mensagem é rejeitada.

10 Reflexão Crítica (Ponto 10)

- **Lamport e Concorrência:** A proporção de $\sim 95\%$ de pares concorrentes com L distinto prova que usar a `vista-lamport.txt` para inferir causalidade leva a falsos positivos.
- **Dimensão do Vetor:** O custo em bytes cresce de forma $O(N)$ (cerca de 16 bytes por carimbo com $N = 3$). Se N mudar dinamicamente, os ficheiros antigos ficam incompatíveis.
- **Entrega Causal:** Exigiria reter registos até as dependências de outras partições estarem satisfeitas. Fica fora do âmbito (secção 2.1).
- **Produtor → Líder:** É impossível detetar perdas no troço, porque o produtor envia sempre $L = 0$ e $V = 0$.